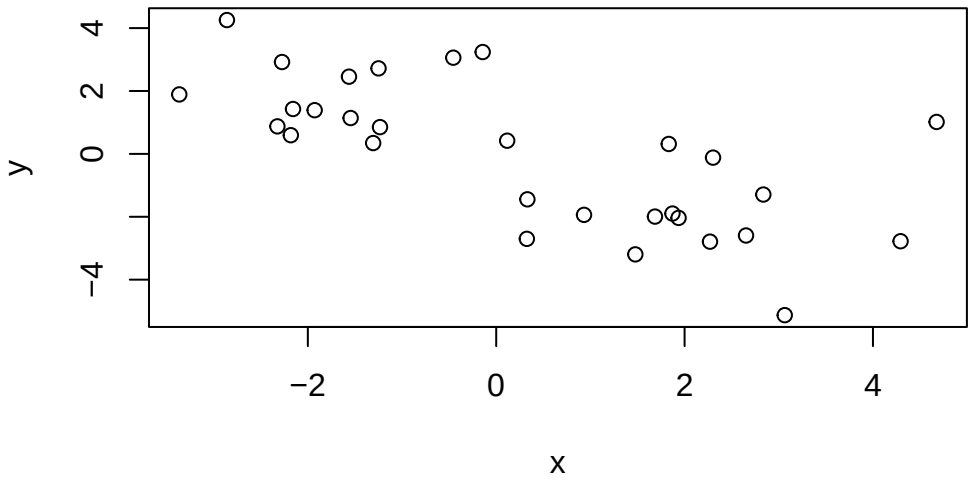
Lab_07

Kenya Sanchez

Generate some data

```
# 15 NUMBERS THAT ARE AROUND -2 OR 2
tmp_x <- c(rnorm(15,-2), rnorm(15,2))
tmp_y <- c(rnorm(15,2), rnorm(15,-2))

# Both vectors become a column of dataframe
# Specify column names
df <- cbind(x=tmp_x, y=tmp_y)
plot(df)
```



Lets do a k-means clustering analysis

```r
# K mean analysis of df with 2 clusters using 20 sandom sets
km <- kmeans(df, centers=2, nstart=20)
km
```

```
K-means clustering with 2 clusters of sizes 15, 15

Cluster means:
          x         y
1  2.165510 -1.905612
2 -1.631238  1.839015

Clustering vector:
 [1] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1

Within cluster sum of squares by cluster:
[1] 53.56432 33.32818
 (between_SS / total_SS =  71.1 %)

Available components:

[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
[6] "betweenss"    "size"         "iter"         "ifault"
```

```r
# Show the structure and components of kmeans object.
attributes(km)
```

```
$names
[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
[6] "betweenss"    "size"         "iter"         "ifault"

$class
[1] "kmeans"
```

Q: Number of points per cluster? Cluster size km$size?

15 points per cluster; Cluster 1 has 15 points and cluster 2 also has 15 points.

```r
km$size
```

```
[1] 15 15
```

Q: Cluster Assignment?

```
km$cluster
```

```
 [1] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

```
# Reveals size in each cluster using Table function
table(km$cluster)
```

```
 1  2
15 15
```

Q: Cluster Centers?

```
km$centers
```

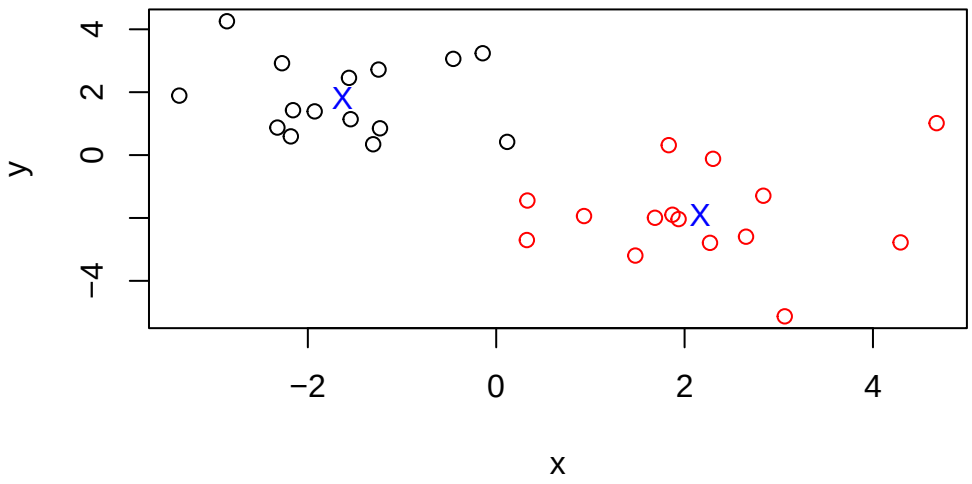
```
          x         y
1  2.165510 -1.905612
2 -1.631238  1.839015
```

```
#One for each cluster
```

Plot x colored by the kmeans cluster assignment and add cluster centers as blue points

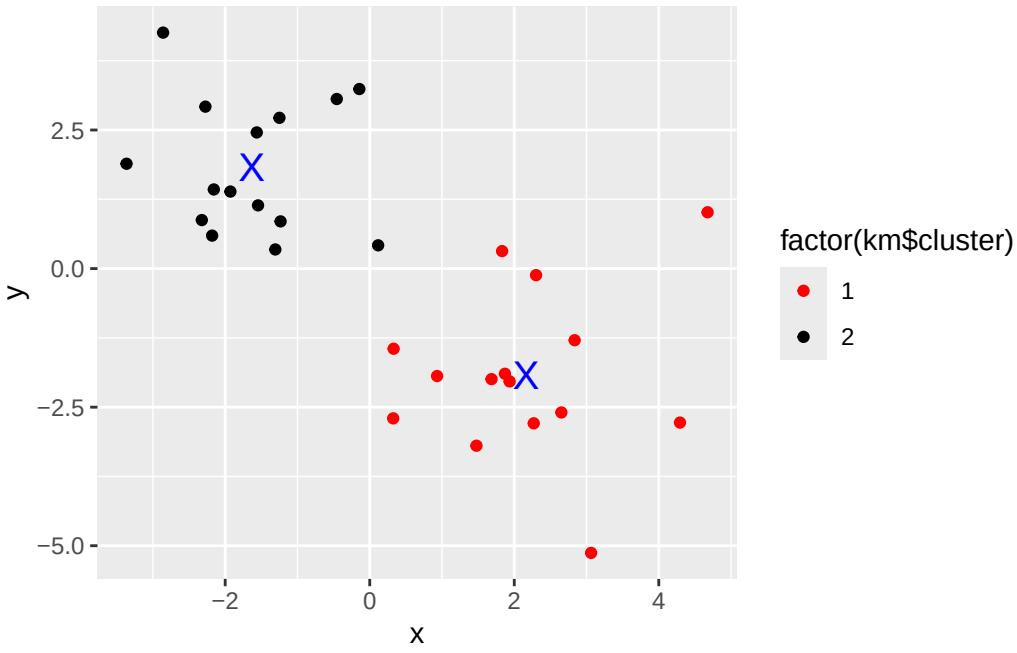
Plot k-means clustering: > Blue X's indicate the centers of each cluster.

```
plot(df, col=c("RED", "BLACK")[km$cluster])
points(km$centers, col="BLUE", pch="X")
```



Plot with ggplot:

```
library(ggplot2)

ggplot(df) +
  aes(x, y, col=factor(km$cluster)) +
  geom_point() +
  scale_color_manual(values=c("RED", "BLACK")) +
  geom_point(data=km$centers, col="BLUE", shape="X", size=5, aes(x,y))
```



Hierarchial Clustering

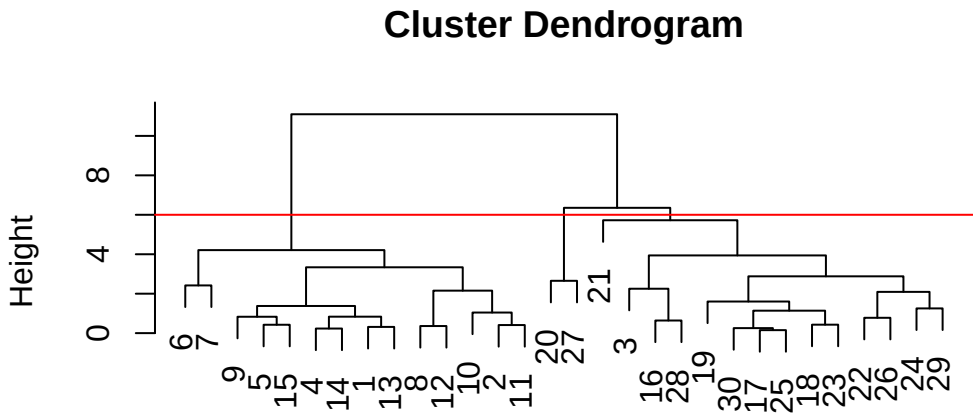
Analyze Hclust()

```
# Name vector as dm and analyze distances in data frame
dm <- dist(df)
```

```
# Create an hclust of the distances in data frame (dm)
hc <- hclust(dm)

hc
```

```
Call:
hclust(d = dm)

Cluster method   : complete
Distance         : euclidean
Number of objects: 30
```

```
# Plot the h-cluster dendrogram

plot(hc)
abline(h=6, col="RED")
```



Lets use Cutree: Cut by height

```
cutree(hc, h=6)
```

```
 [1] 1 1 2 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 3 2 2 2 2 2 2 3 2 2 2
```

```
table(cutree(hc,h=6))
```

```
 1  2  3
14 14  2
```

Cut by K

```
# Cut by k=2 ,eans cut so there are 2 clusters
cutree(hc, k=2)
```

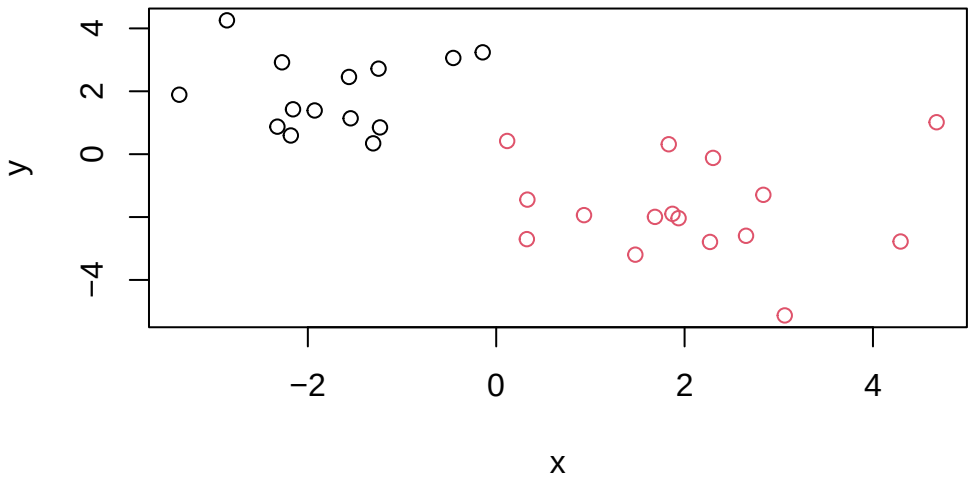
```
 [1] 1 1 2 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
```

4 clusters

```
cutree(hc, k=4)
```

```
 [1] 1 1 2 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 3 4 2 2 2 2 2 3 2 2 2
```

```
# Plot the 2 clusters generated in Cutree
plot(df, col=cutree(hc, k=2))
```



Lab_07 - April 20th, 2026

Start of Hands on Section

Section 1: PCA of UK food data

Data Import

```
url <- "https://tinyurl.com/UK-foods"
x <- read.csv(url)
```

Q1: How many rows and columns are in your new data frame named x? What R functions could you use to answer this questions?

Answer Q1: There are 17 rows and 5 columns in this new data frame. The R functions used to answer this question are shown below

```
# Number of Rows ONLY
nrow(x)
```

```
[1] 17
```

```
# Number of Columns ONLY
ncol(x)
```

```
[1] 5
```

```
# Number of Rows AND Columns
dim(x)
```

```
[1] 17  5
```

Checking your data

Preview of first 6 rows

```
# Preview for the first 6 rows

head(x)
```

```
              X England Wales Scotland N.Ireland
1        Cheese     105   103      103        66
2  Carcass_meat     245   227      242       267
3    Other_meat     685   803      750       586
4          Fish     147   160      122        93
5 Fats_and_oils     193   235      184       209
6        Sugars     156   175      147       139
```

Fixing Number of Columns

```
# Removing rownames from being a column
rownames(x) <- x[,1]
x <- x[,-1]
head(x)
```

```
               England Wales Scotland N.Ireland
Cheese             105   103      103        66
Carcass_meat       245   227      242       267
Other_meat         685   803      750       586
Fish               147   160      122        93
Fats_and_oils      193   235      184       209
Sugars             156   175      147       139
```

```
# Total of 4 columns only
```

Check dimensions again:

```
# Verifying that rownames is no longer a column

dim(x)
```

```
[1] 17  4
```

Alternative Approach

Done during reading of data file

```
# Removes row names form being a column during data fiel read

x <- read.csv(url, row.names=1)
head(x)
```

```
               England Wales Scotland N.Ireland
Cheese             105   103      103        66
Carcass_meat       245   227      242       267
Other_meat         685   803      750       586
Fish               147   160      122        93
Fats_and_oils      193   235      184       209
Sugars             156   175      147       139
```

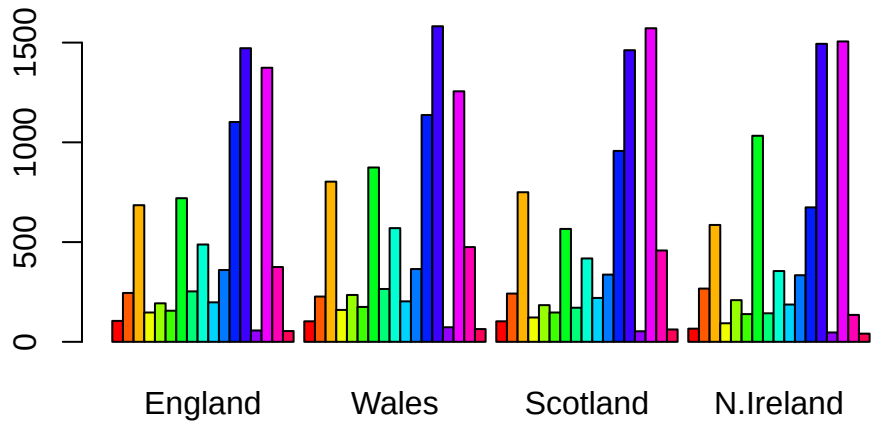
Q2: Which approach to solving the 'row-names problem' mentioned above do you prefer and why? Is one approach more robust than another under certain circumstances?

Answer Q2: I prefer the alternative method of removing the rownames column as the data is being imported and read because it allows us to get start analyzing without having to make multistep corrections. The second 'alternative' approach is more robust than the the first because it correctly assigns the first column in the first step and does not interfere with the display of the dataset. On the other hand, the x <- x[,-1] approach is still effective but if this is done more than once, we risk the possibility of losing the next column from of data set.

Spotting major differences and trends

Generating Bar Plots

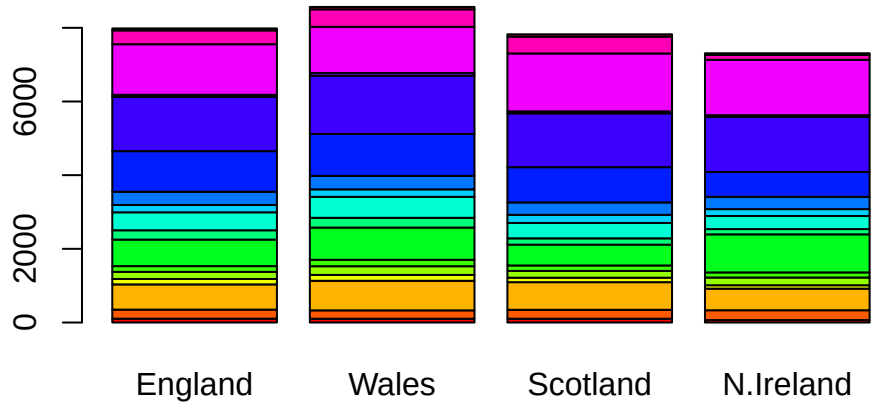
```
# Using R base, not ggplot
barplot(as.matrix(x), beside=T, col=rainbow(nrow(x)))
```



Q3: Changing what optional argument in the above barplot() function results in the following plot? stacked version of barplot

Answer Q3: Changin the bedside arguemnt in the barplot() function above results in a stacked boxplot rather than bars of the boxplot being shown side by side. As shown below, I altered the bedside funciton in abrplot() and made this equal to being FALSE. This removed the side by side structure and kept every barplot stacked.

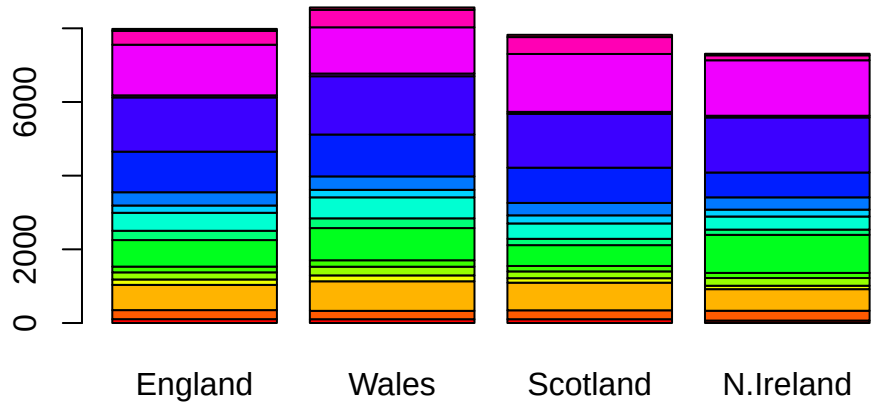
```
barplot(as.matrix(x), beside=FALSE, col=rainbow(nrow(x)))
```



Leave Bedside Argument Out:

> Leaving the bedside argument out results the same as setting it to false because "false" is the default version in which barplots are displayed.

```
# Default Bar Plot
barplot(as.matrix(x), col=rainbow(nrow(x)))
```



Use long format and tidyr package

```
library(tidyr)

# Convert data to long format for ggplot with `pivot_longer()`

x_long <- x |>
          tibble::rownames_to_column("Food") |>
          pivot_longer(cols = -Food,
                       names_to = "Country",
                       values_to = "Consumption")

dim(x_long)
```

```
[1] 68  3
```

Preview of “Long” table

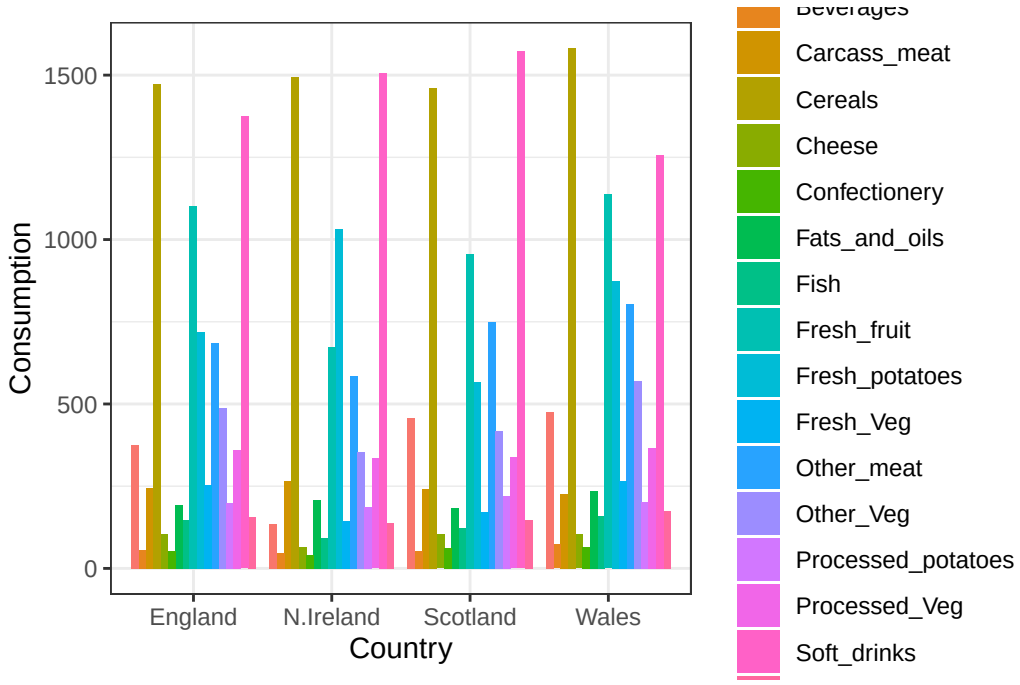
```
head(x_long)
```

```
# A tibble: 6 x 3
  Food            Country   Consumption
```

```
  <chr>           <chr>           <int>
1 "Cheese"        England           105
2 "Cheese"        Wales             103
3 "Cheese"        Scotland          103
4 "Cheese"        N.Ireland          66
5 "Carcass_meat " England           245
6 "Carcass_meat " Wales             227
```

Barplot using ggplot

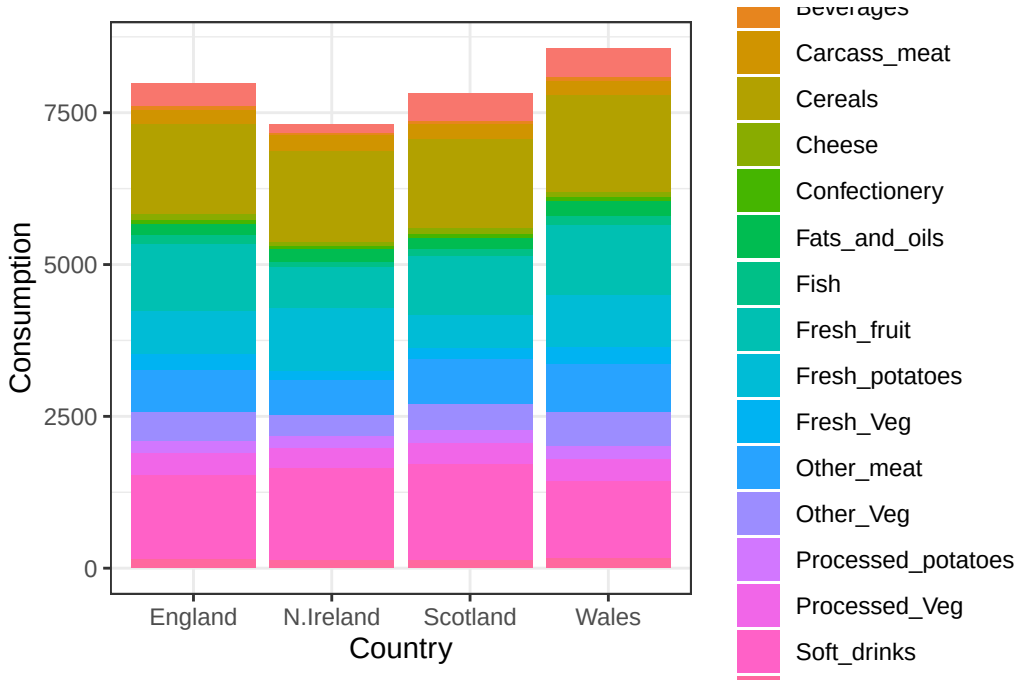
```
library(ggplot2)

ggplot(x_long) +
  aes(x = Country, y = Consumption, fill = Food) +
  geom_col(position = "dodge") +
  theme_bw()
```



Q4: Changing what optional argument in the above ggplot() code results in a stacked barplot figure?

Answer Q4: Changing the geom_col() argument changes the display of the columns. We would have to remove the dodge feautre in the position. As we saw previously, the deafault setting is to have a stacked barplot. Correct R code shown below:

```
ggplot(x_long) +
  aes(x = Country, y = Consumption, fill = Food) +
  geom_col() +
  theme_bw()
```

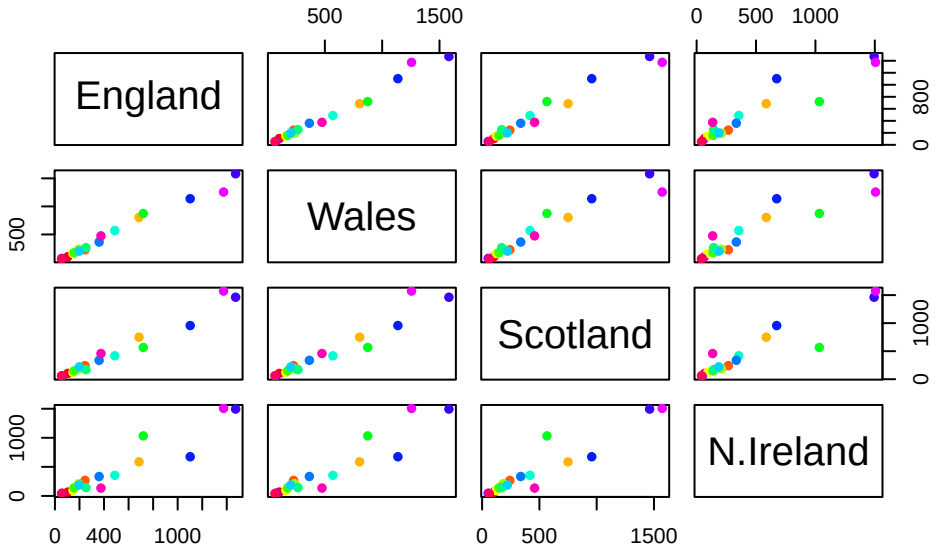


Pairs plots and heatmaps

Q5: We can use the pairs() function to generate all pairwise plots for our countries. Can you make sense of the following code and resulting figure? What does it mean if a given point lies on the diagonal for a given plot? *pairs(x, col=rainbow(nrow(x)), pch=16)

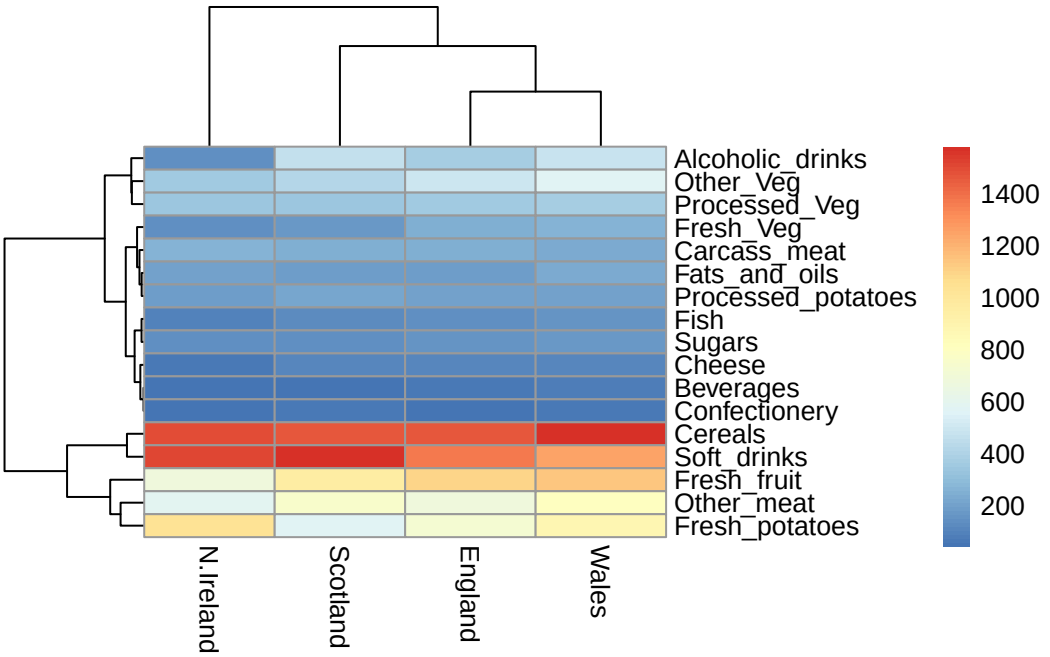
> **Answer Q5:** Using the pairs() function results in a matrix of scatterpltos where every country is plotted against every other country. If a given point lies on the diagonal for a given plot this means that the country has approximately the same value as the other it is compaired to. Points farther from this diagonal indicate more differneces between the countries being compared.

```
pairs(x, col=rainbow(nrow(x)), pch=16)
```



Generate Heat Map

```
library(pheatmap)

pheatmap( as.matrix(x) )
```



Q6. Based on the pairs and heatmap figures, which countries cluster together and what does this suggest about their food consumption patterns? Can you easily tell what the main differences between N. Ireland and the other countries of the UK in terms of this data-set?

Answer Q6: Based on the pairs and hetmap, Engaland and Wales cluster together the best. The country that clusters the "worst" is N. Ireland as it is the least "digaonal" in all of the pair plots. Although Scotland does not cluster as well in the heatmap, it clusters better diagonally in the pair plot. This cluster suggest that the food consumption patterns of beverages and many other items follow a nearly identical trend. When the points are diagonally organized on the pair plot this means that the countries compared consume nearly the exact same amount. Although it is relatively easy to see that Northern Ireland is different (especially when looking at the pair plot), it is difficult to extract the specific reasons from the heatmap alone because of the scale. The colors fall so close to each other that the differnces appear minimal. Overall, it is easy to identify that N. Ireland has a different consumption patter in regards to items like alcoholic bevarages and the amount of consumption based on both the heat map and the pair plot.

PCA to the rescue

Prcomp() function expects the observations to be rows and the variables to be columns therefore we need to first transpose our data

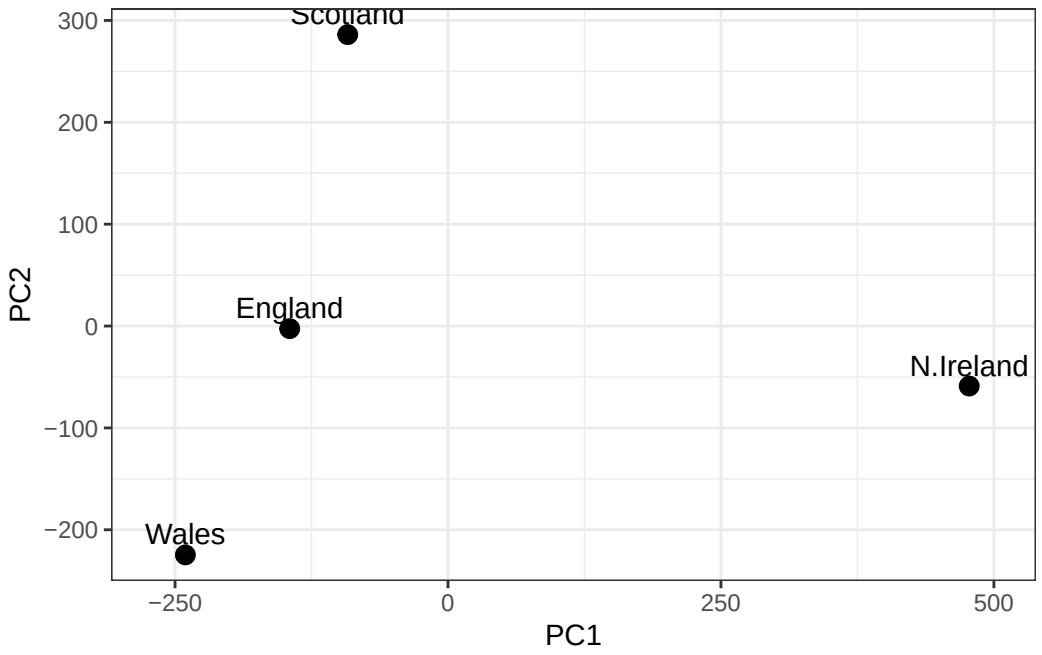
```
# Use the prcomp() PCA function
pca <- prcomp( t(x) )
summary(pca)
```

```
Importance of components:
                            PC1      PC2      PC3       PC4
Standard deviation     324.1502 212.7478 73.87622 2.921e-14
Proportion of Variance   0.6744   0.2905  0.03503 0.000e+00
Cumulative Proportion    0.6744   0.9650  1.00000 1.000e+00
```

Q7. Complete the code below to generate a plot of PC1 vs PC2. The second line adds text labels over the data points.

> **Answer Q7:** The code below was completed by adding labels to x and y in aesthetics to ensure the correct plotting of this graph. Shown below is the completed code and corresponding plot of PC1 vs. PC2.

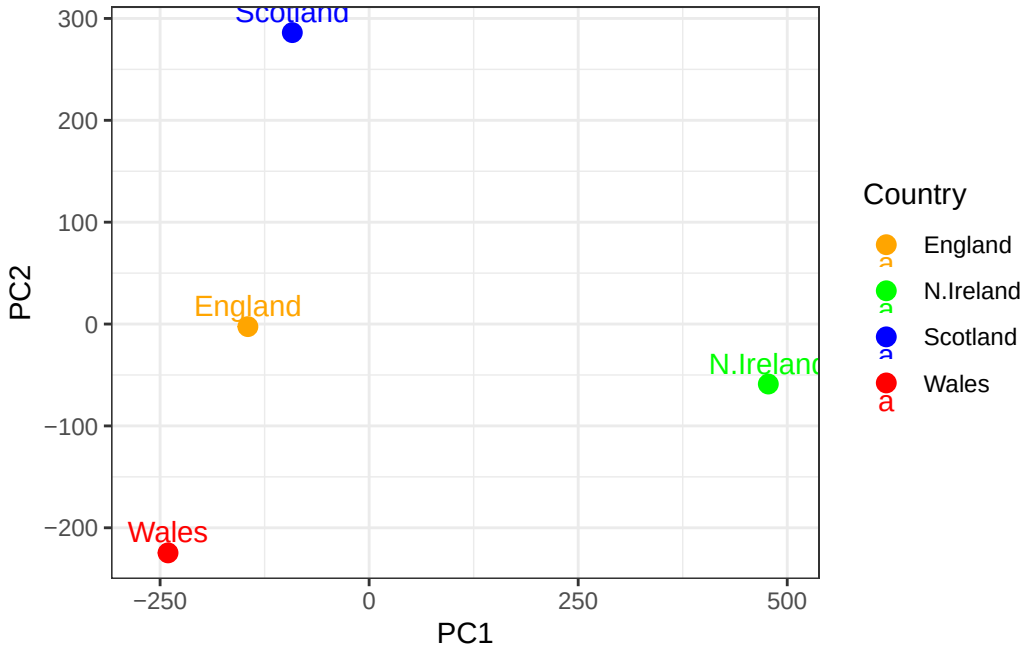
```
# Create a data frame for plotting
df <- as.data.frame(pca$x)
df$Country <- rownames(df)

# Plot PC1 vs PC2 with ggplot
ggplot(pca$x) +
  aes(x = PC1, y = PC2, label = rownames(pca$x)) +
  geom_point(size = 3) +
  geom_text(vjust = -0.5) +
  xlim(-270, 500) +
  xlab("PC1") +
  ylab("PC2") +
  theme_bw()
```



Q8: Customize your plot so that the colors of the country names match the colors in our UK and Ireland map and table at start of this document.

> **Answer Q8:** In order to plot colors based on country names we had to assing a specific color to the country and include the scale_color_manual funciton. Shown below is the completed R code for the country colors and its corresponding plot.

```
ggplot(df) +
  aes(x = PC1, y = PC2, label = Country, col=Country) +
  geom_point(size = 3) +
  geom_text(vjust = -0.5) +
  xlim(-270, 500) +
  xlab("PC1") +
  ylab("PC2") +
  theme_bw() +
  scale_color_manual(values = c(Wales="red", N.Ireland="green", Scotland="blue", England="ora
```



Below we can use the square of pca$sdev ("standard deviation") to calculate how much variation in the original data each PC accounts for.

```
# Variation in Original Data for Each PC
v <- round( pca$sdev^2/sum(pca$sdev^2) * 100 )
v
```

```
[1] 67 29  4  0
```

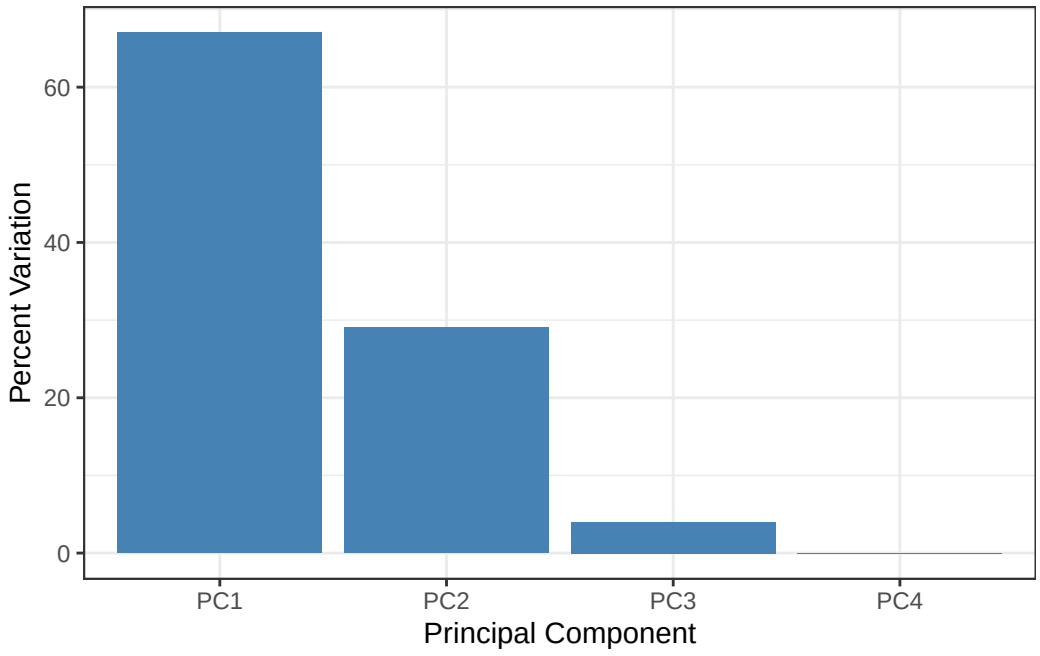
```
## Alternative method to find variance is second row of the code below
z <- summary(pca)
z$importance
```

```
                             PC1       PC2      PC3          PC4
Standard deviation     324.15019 212.74780 73.87622 2.921348e-14
Proportion of Variance   0.67444   0.29052  0.03503 0.000000e+00
Cumulative Proportion    0.67444   0.96497  1.00000 1.000000e+00
```

Above, The Proportion of Variance is the standard deviation is decimal form for its corresponding PC.

Generate a Plot for the %Variance of each PC

```r
# Create scree plot with ggplot
variance_df <- data.frame(
  PC = factor(paste0("PC", 1:length(v)), levels = paste0("PC", 1:length(v))),
  Variance = v
)

ggplot(variance_df) +
  aes(x = PC, y = Variance) +
  geom_col(fill = "steelblue") +
  xlab("Principal Component") +
  ylab("Percent Variation") +
  theme_bw() +
  theme(axis.text.x = element_text(angle = 0))
```

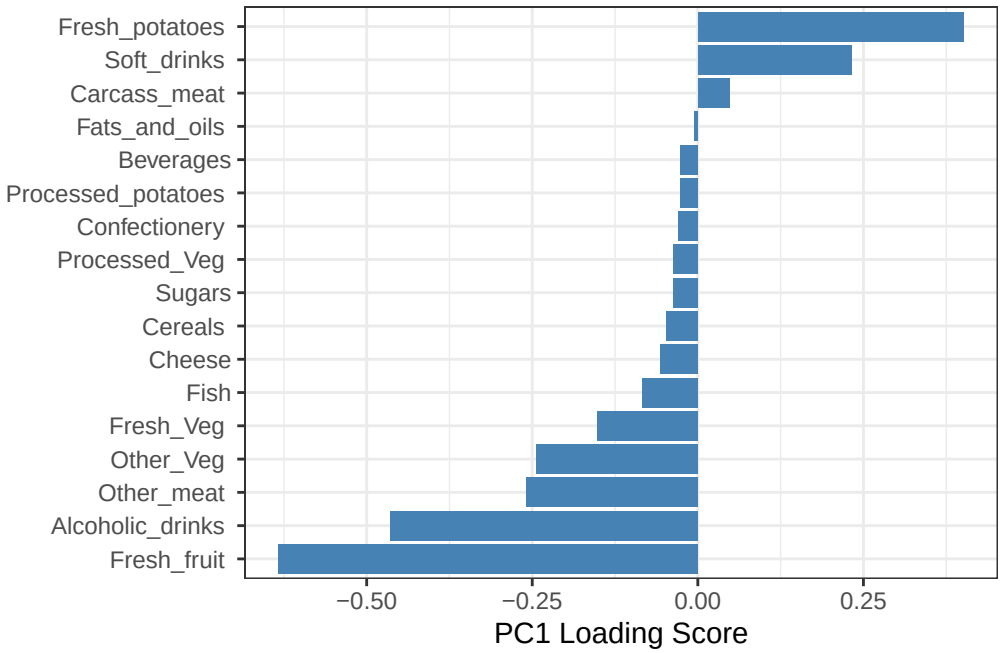


Digging deeper (variable loadings)

Loading Score

```r
## Lets focus on PC1 as it accounts for > 90% of variance
ggplot(pca$rotation) +
  aes(x = PC1,
```

```
      y = reorder(rownames(pca$rotation), PC1)) +
  geom_col(fill = "steelblue") +
  xlab("PC1 Loading Score") +
  ylab("") +
  theme_bw() +
  theme(axis.text.y = element_text(size = 9))
```

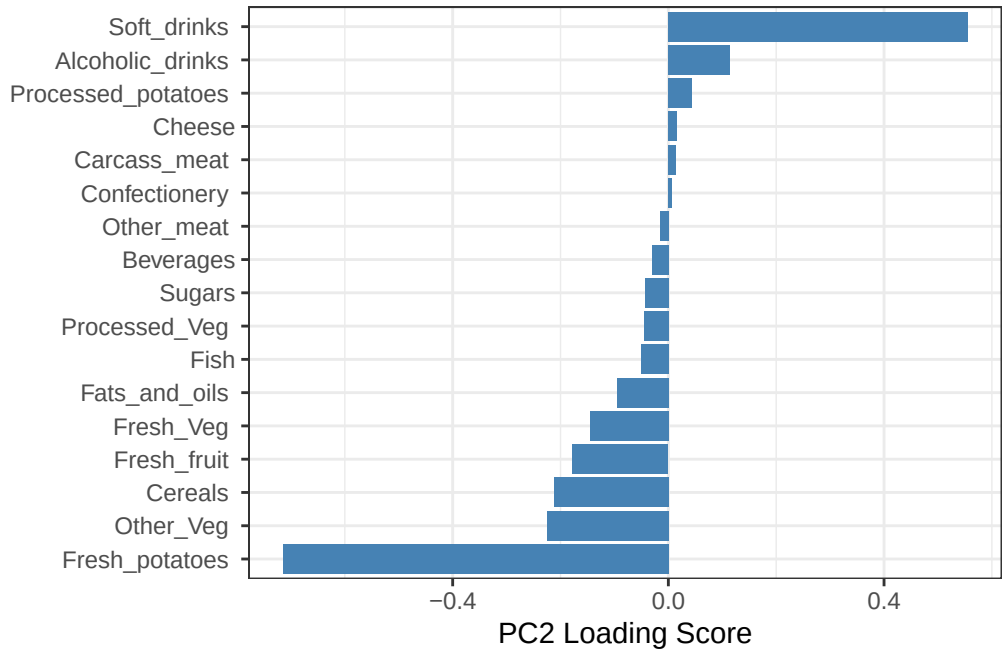


Q9: Generate a similar 'loadings plot' for PC2. What two food groups feature prominantely and what does PC2 maninly tell us about?

> **Answer Q9:** The two food groups that are prominantely featured are Fresh Potatoes and Soft Drinks. PC1 was in chared of separating N.Ireland from the rest of the countries (Wales, England, and Scotland). On the other hand, PC2 compares Scotland to England and Wales rather than to N. Ireland. In other words, PC1 captured the majority of the differences in the entire dataset(N. Ireland vs.all other Countries" and PC2 cpatured the "next" most significant pattern between the remaining 3 countries.

```
# Plot Loading Score for PC2
ggplot(pca$rotation) +
  aes(x = PC2,
      y = reorder(rownames(pca$rotation), PC2)) +
  geom_col(fill = "steelblue") +
```

```
  xlab("PC2 Loading Score") +
  ylab("") +
  theme_bw() +
  theme(axis.text.y = element_text(size = 9))
```



Section 2: PCA of RNA-seq data (Optional)

Loading Data from URL

```
url2 <- "https://tinyurl.com/expression-CSV"
rna.data <- read.csv(url2, row.names=1)
head(rna.data)
```

```
       wt1 wt2  wt3  wt4 wt5 ko1 ko2 ko3 ko4 ko5
gene1  439 458  408  429 420  90  88  86  90  93
gene2  219 200  204  210 187 427 423 434 433 426
gene3 1006 989 1030 1017 973 252 237 238 226 210
gene4  783 792  829  856 760 849 856 835 885 894
gene5  181 249  204  244 225 277 305 272 270 279
gene6  460 502  491  491 493 612 594 577 618 638
```

Q10: How many genes and samples are in this data set? How many PCs do you think it will take to have a useful overview of this data set (see below)?

> **Answer Q10:** There are 100 rows and 10 columns thus meaning that there are 100 genes and 10 samples.

```
dim(rna.data)
```

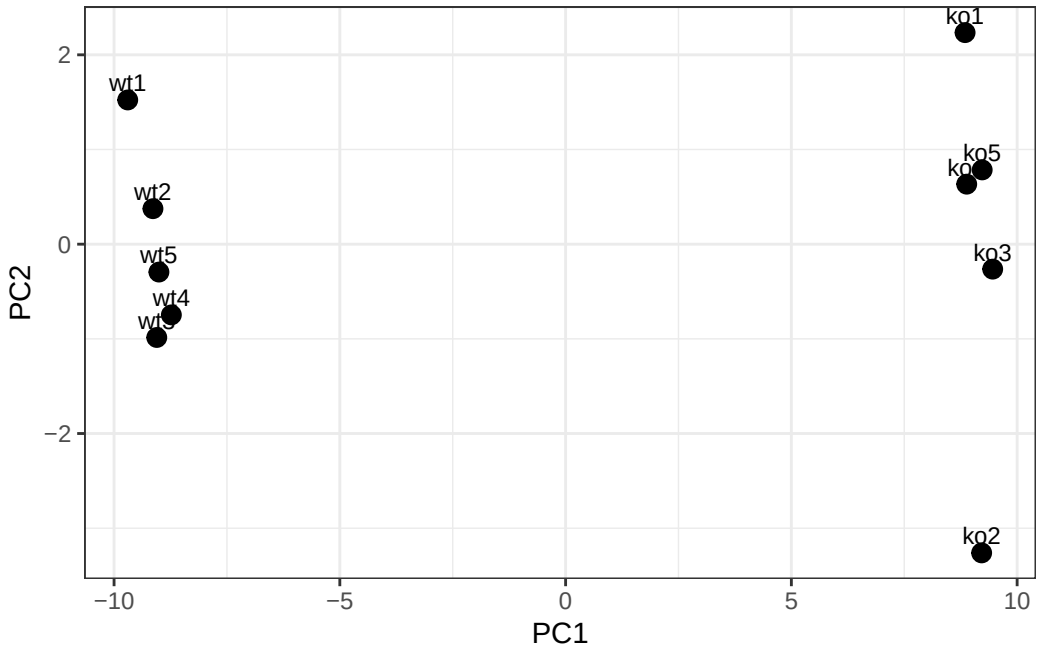
```
[1] 100  10
```

Generating Barplot()

```
## Again we have to take the transpose of our data
pca <- prcomp(t(rna.data), scale=TRUE)
```

```
# Create data frame for plotting
df <- as.data.frame(pca$x)
df$Sample <- rownames(df)
```

```
## Plot with ggplot
library(ggplot2)

ggplot(df) +
  aes(x = PC1, y = PC2, label = Sample) +
  geom_point(size = 3) +
  geom_text(vjust = -0.5, size = 3) +
  xlab("PC1") +
  ylab("PC2") +
  theme_bw()
```



Summary of how much variation in the original data each PC accounts for:

```
summary(pca)
```

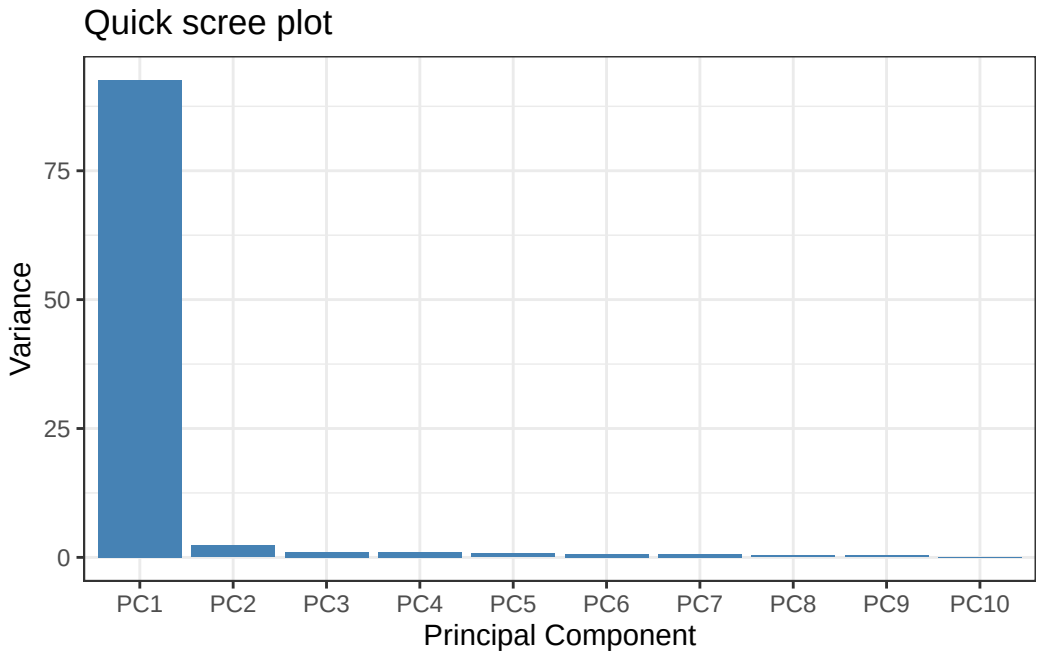
```
Importance of components:
                          PC1    PC2     PC3     PC4     PC5     PC6     PC7
Standard deviation     9.6237 1.5198 1.05787 1.05203 0.88062 0.82545 0.80111
Proportion of Variance 0.9262 0.0231 0.01119 0.01107 0.00775 0.00681 0.00642
Cumulative Proportion  0.9262 0.9493 0.96045 0.97152 0.97928 0.98609 0.99251
                           PC8     PC9    PC10
Standard deviation     0.62065 0.60342 3.3e-15
Proportion of Variance 0.00385 0.00364 0.0e+00
Cumulative Proportion  0.99636 1.00000 1.0e+00
```

PC1 captures 92.6% of the original variance with the first two PCs capturing 94.9%

Scree Plot Summary

```
# Calculate variance explained
pca.var <- pca$sdev^2
pca.var.per <- round(pca.var/sum(pca.var)*100, 1)

# Create scree plot data
```

```r
scree_df <- data.frame(
  PC = factor(paste0("PC", 1:10), levels = paste0("PC", 1:10)),
  Variance = pca.var[1:10]
)

ggplot(scree_df) +
  aes(x = PC, y = Variance) +
  geom_col(fill = "steelblue") +
  ggtitle("Quick scree plot") +
  xlab("Principal Component") +
  ylab("Variance") +
  theme_bw()
```



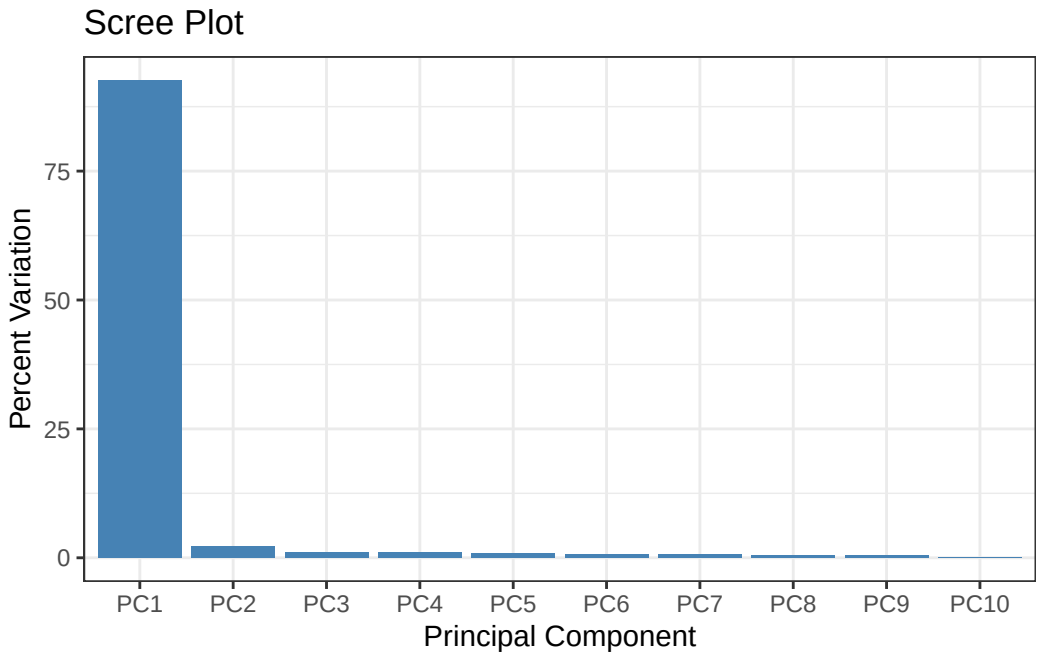
Alternative Method to Obtain Percent Variance

```r
## Percent variance is often more informative to look at
pca.var.per
```

```
 [1] 92.6  2.3  1.1  1.1  0.8  0.7  0.6  0.4  0.4  0.0
```

Alternative Scree Plot

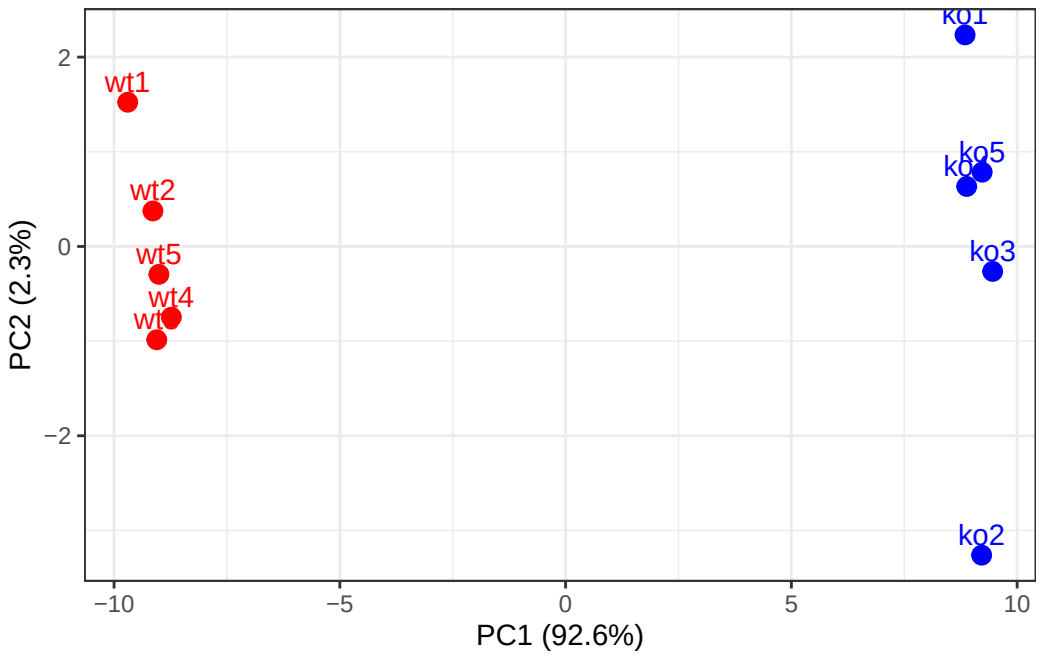
```r
# Create percent variance scree plot
scree_pct_df <- data.frame(
  PC = factor(paste0("PC", 1:10), levels = paste0("PC", 1:10)),
  PercentVariation = pca.var.per[1:10]
)

ggplot(scree_pct_df) +
  aes(x = PC, y = PercentVariation) +
  geom_col(fill = "steelblue") +
  ggtitle("Scree Plot") +
  xlab("Principal Component") +
  ylab("Percent Variation") +
  theme_bw()
```



Improved PCA Plot

```r
## A vector of colors for wt and ko samples
colvec <- colnames(rna.data)
colvec[grep("wt", colvec)] <- "red"
colvec[grep("ko", colvec)] <- "blue"

# Add condition to data frame
df$condition <- substr(df$Sample, 1, 2)
```

```
df$color <- colvec

ggplot(df) +
  aes(x = PC1, y = PC2, color = color, label = Sample) +
  geom_point(size = 3) +
  geom_text(vjust = -0.5, hjust = 0.5, show.legend = FALSE) +
  scale_color_identity() +
  xlab(paste0("PC1 (", pca.var.per[1], "%)")) +
  ylab(paste0("PC2 (", pca.var.per[2], "%)")) +
  theme_bw()
```



Gene Loadings:

```
loading_scores <- pca$rotation[,1]

## Find the top 10 measurements (genes) that contribute
## most to PC1 in either direction (+ or -)
gene_scores <- abs(loading_scores)
gene_score_ranked <- sort(gene_scores, decreasing=TRUE)

## show the names of the top 10 genes
top_10_genes <- names(gene_score_ranked[1:10])
top_10_genes
```

```
 [1] "gene100" "gene66"  "gene45"  "gene68"  "gene98"  "gene60"  "gene21"
 [8] "gene56"  "gene10"  "gene90"
```

This indicates the top 10 genes that affect PC1